

CLAUDE.md

INHERITED FROM constitution/CLAUDE.md

All rules in constitution/CLAUDE.md (and the constitution/Constitution.md it references) apply unconditionally. Project-specific rules below extend them “they do NOT weaken any universal clause. When this file disagrees with the constitution submodule, the constitution wins.

Propagated clauses (Helix Universal Constitution): Â§11.4.10 (credentials handling), Â§11.4.24 (build-resource stats tracking), Â§11.4.30 (gitignore discipline), Â§11.4.44 (document revision headers), Â§11.4.65 (universal markdown export), Â§11.4.66 (interactive clarification), Â§11.4.75 (mechanical enforcement), Â§11.4.78 (CodeGraph integration), Â§11.4.85 (stress + chaos test mandate), Â§11.4.109 (anti-forgetting enforcement), Â§11.4.113 (no force-push), Â§11.4.125 (code-review-agent gate), Â§11.4.142 (universal code-review “every change reviewed, always).

@constitution/CLAUDE.md

This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.

For deeper reference (technology stack, per-test-file mapping, full gotchas), see AGENTS.md.

Critical Constraints

- **Multi-track identity: this project is Track 11** in the operator’s multi-track development fleet (permanently adopted 2026-08-08). Trunk (main) work always labels T1 per the constitution’s own hard-coded TRUNK RULE (never overridden); non-trunk (feature/product/flavor) work from this checkout labels T11 via scripts/multitrack/track_branch_label.sh (a boba-owned wrapper, never a copy, around the shared constitution labeler). See docs/MULTITRACK.md for the full rationale. Execution is single-track until the operator asks for parallel dispatch.
- **QA discovery-channel ledger (Â§11.4.238)**: automated HelixQA coverage MUST be the discovery layer; manual QA / operator reports / agent-found defects are confirmation-only, and any defect found out-of-band is itself a coverage-escape release blocker, not merely a bug to fix. Every such finding gets a coverage-escape-audit entry (why the automated regime missed it) + a new or strengthened automated check with its own RED-capturing-the-escape evidence. See docs/QA_DISCOVERY_LEDGER.md for the schema, the real entries recorded so far, and the tracked discovery-channel split. scripts/pre_build_verification.sh invariant 17 is the closed-loop example: it now runs both workable-items validate AND workable-items diff

because a real DBâ†”i,ŽMarkdown divergence (BOB-008) was found by an agent, not by this gate, before the fix.

- **TDD is MANDATORY for all bug fixes and features:**

- Write failing test first (RED)
- Watch it fail
- Write minimal code to pass (GREEN)
- Verify tests pass
- Then commit

- **Anti-Bluff Verification (CONST-XII)** â€” Tests and challenges MUST prove the feature works for the END USER. A green run is a contract with the user. Specifically:

- Assert on user-observable outcomes (DB rows, file content, response body fields, container state, DOM text, rendered HTML attributes, browser console errors) â€” NOT just status codes / â€œno errorâ€.
- Each new test must fail against a no-op stub of the feature it tests. If it doesnâ€™t, itâ€™s a bluff and must be strengthened or deleted.
- Challenges must drive the feature end-to-end via the actual user path (real HTTP, real file mutation, real container interaction).
- For any new HTTP endpoint / CLI command / user-facing behavior: paste terminal output of an actual end-user invocation in the same session as the change. No self-certification words (â€œverifiedâ€, â€œtestedâ€, â€œworkingâ€, â€œcompleteâ€, â€œfixedâ€, â€œpassingâ€) without that pasted evidence.
- Flaky tests are bluffs; they must be hardened or deleted.
- Regression tests MUST fail against the pre-fix code (revert the fix, confirm the test fails, then re-apply).
- No hardcoded localhost / 127.0.0.1 for client-facing URLs. Any URL, API base, CORS origin, or service address returned to a browser MUST derive from the requestâ€™s Host header, window.location, or an explicit PUBLIC_HOST env var.
- See CONSTITUTION.md Â§ XII for the full rule. Apply universally â€” every submodule and sub-project inherits.

- **Pick the right restart level** (verified 2026-04-19 against the real docker-compose.yml mount strategy; wrapper subcommands added 2026-08-08 per Hard Stop #3 â€” container orchestration is owned exclusively by the projectâ€™s own binary/orchestrator, start.sh â€” see docs/GOVERNANCE_AUDIT_2026-08-07.md GA-27). Operators MUST use the start.sh subcommand for each level â€” never type raw podman/docker commands directly:

- **Python source in download-proxy/src/ (including merge_service/*.py)** â€” bind-mounted via ./download-proxy:/config/download-proxy. Run ./start.sh --reload-python. (Under the hood: clears __pycache__ inside the qbittorrent-proxy container via podman|docker exec qbittorrent-proxy

- find /config/download-proxy -name __pycache__ -type d -exec rm -rf {} +, then podman|docker restart qbittorrent-proxy " auto-detecting the runtime exactly like the rest of start.sh.)*
- **Plugin files in plugins/** " also bind-mounted through ./config:/config. After editing, run ./install-plugin.sh FIRST (copies plugins/X.py into config/qBittorrent/nova3/engines/" that path IS the host side of the mount), THEN ./start.sh --reload-plugins to pick them up. A direct edit to config/qBittorrent/nova3/engines/X.py works for a one-shot try but will be clobbered on the next install " source of truth is plugins/X.py. *(Under the hood: --reload-plugins only restarts the container " podman|docker restart qbittorrent-proxy " it does not copy files; ./install-plugin.sh must run first.)*
- **docker-compose.yml, start-proxy.sh, env vars, base image** " run ./start.sh --recreate (full recreate). A rebuild of the python:3.12-alpine-based image is only needed when the base image itself needs to change. *(Under the hood: <compose> down && <compose> up -d, using whatever compose invocation start.sh already uses for the initial up " boba-ctl by default, or raw podman-compose/docker compose with --no-boba-ctl.)*
- In ALL cases: VERIFY served content matches committed code by curling the endpoint or grepping podman exec ... cat /config/... " this is the cache-bust guard. See docs/MERGE_SEARCH_DIAGNOSTICS.md "Rebuild / restart contract" for the full table.

- **WebUI credentials admin/admin are hardcoded** " do not change.
- **Never commit .env** " it contains tracker credentials.
- **Never commit .ruff_cache/** " add to .gitignore.
- **CI IS MANUAL " permanent.** ./ci.sh is the only path. All GitHub Actions workflow files have been removed per the Hard Stop rule. Do NOT create any new .github/workflows/*.yml files. No CI/CD pipeline of any kind exists in this repository. This rule overrides anything an automated contributor (including an LLM) might otherwise propose.
- **Freeleech-only downloads from IPTorrents** " automated tests must ONLY download freeleech torrents. Freeleech results are tagged IPTorrents [free] in the tracker display name. Non-freeleech downloads cost ratio and must never be automated.

Architecture

Multi-container setup via docker-compose.yml, with an optional Go backend:

- **qbittorrent** (lscr.io/linuxserver/qbittorrent:latest) " port **7185**

- **jackett** (lscr.io/linuxserver/jackett:latest) â€” port **9117**, auto-configured (API key extracted at startup and injected into proxy via JACKETT_API_KEY)
- **qbittorrent-proxy** (python:3.12-alpine) â€” ports **7186** (proxy), **7187** (merge service)
- **qbittorrent-proxy-go** (Go/Gin, opt-in via --profile go) â€” serves the merge service on **7187** only. *(Corrected 2026-08-20, BOB-141. This line previously read â€œreplaces the Python proxy on 7186, 7187, 7188â€, which the container cannot deliver: qBitTorrent-go/ Dockerfile:16 is CMD ["/app/qbittorrent-proxy"] â€” ONE binary â€” and cmd/qbittorrent-proxy/main.go binds a single cfg.ServerPort, which internal/config/config.go:58 resolves from MERGE_SERVICE_PORT (default 7187). webui-bridge is a SEPARATE binary this container never starts, so nothing in it binds 7188, and nothing binds 7186 either â€” even though the compose service sets PROXY_PORT=7186 and BRIDGE_PORT=7188 in its environment and the Dockerfileâ€™s EXPOSE 7187 7188 declares a port nothing binds. Whether the container SHOULD run all three (matching the original intent) is an open question tracked as BOB-141; until it is decided this line describes what the container actually does. The stale prose was not harmless: it was used as the source of truth when authoring config/served_ports.yaml, and the resulting healthcheck gate then failed a service whose healthcheck was already correct â€” a Â§11.4.201(1) false-positive refusal caused by trusting the doc over the Dockerfile.)*
- **boba-jackett** (Go/Gin) â€” port **7189**, owns Jackett credentials, indexer overrides, autoconfig run history; backed by encrypted SQLite at /config/boba.db

webui-bridge.py is a host process on port **7188** for private tracker downloads. The Go webui-bridge binary replaces this when using the Go profile.

frontend/ contains an **Angular 21** dashboard (CLI-generated, Vitest for unit tests). Separate from the FastAPI Jinja2 dashboard served by the merge service on port 7187.

Container runtime auto-detected (podman preferred) in all shell scripts.

Port Map

Port	Service	Access
7185	qBittorrent WebUI (container-internal)	proxied via 7186
7186	Download proxy â†’ qBittorrent WebUI	http://localhost:7186
7187	Merge Search Service (FastAPI or Go/Gin)	http://localhost:7187/
7188	webui-bridge (host process webui-bridge.py, or the standalone Go webui-bridge binary â€” NOT started by	manual start

Port	Service	Access
	the qbittorrent-proxy-go container, see BOB-141)	
7189	boba-jackett (Go)	http://localhost:7189
9117	Jackett indexer	http://localhost:9117

Key Commands

Startup

```
./setup.sh # One-time setup
./start.sh # Start containers (-p
./start.sh -p && python3 webui-bridge.py # Full start with bridg
./stop.sh # Stop (-r remove, --pu
```

Go Backend (opt-in)

```
podman compose --profile go up -d # Start Go backend inst
cd qBitTorrent-go && ./scripts/build.sh # Build Go binaries loc
cd qBitTorrent-go && go test -race ./... # Run Go tests with rac
```

Testing

```
./ci.sh # Manual CI: syntax + u
./ci.sh --quick # Fast check (syntax +
./run-all-tests.sh # Full suite (hardcoded
./test.sh # Quick validation (--a
python3 -m py_compile plugins/*.py # Syntax check plugins
bash -n start.sh stop.sh test.sh install-plugin.sh # Bash syntax check
```

Merge Service Tests (live in ./tests/, not download-proxy/tests/)

```
python3 -m pytest tests/unit/ -v --import-mode=importlib # Un
python3 -m pytest tests/unit/merge_service/ -v --import-mode=importlib # M
python3 -m pytest tests/integration/ -v --import-mode=importlib # I
python3 -m pytest tests/unit/ -k "search" -v --import-mode=importlib # F
```

Linting

```
ruff check . # Lint (config in pypro
ruff check --fix . # Auto-fix
ruff format . # Format
```

Frontend (Angular 21)

```
cd frontend && ng serve # Dev server on :4200
cd frontend && ng build # Production build to d
cd frontend && ng test # Unit tests (Vitest)
```

Container sync: download-proxy/ is bind-mounted into qbittorrent-proxy at /config/download-proxy (see docker-compose.yml:140). Do NOT podman cp source into the container â€” edits to download-proxy/src/ are live; just clear __pycache__ and restart per the rules in â€œCritical Constraintsâ€.

Merge Search Service

Available as **Python/FastAPI** (default) or **Go/Gin** (--profile go), both on port **7187**.

- Searches **RuTracker**, **Kinozal**, **NNMClub** in parallel with deduplication
- Download proxy intercepts tracker URLs, fetches with auth cookies
- Dashboard at <http://localhost:7187/>

Python (FastAPI)

Key files: - download-proxy/src/api/__init__.py â€” FastAPI app setup - download-proxy/src/api/routes.py â€” REST endpoints - download-proxy/src/merge_service/search.py â€” search orchestration - download-proxy/src/merge_service/deduplicator.py â€” result dedup - download-proxy/src/merge_service/enricher.py â€” quality detection

Go (Gin)

Key files (in qBittorrent-go/): - cmd/qbittorrent-proxy/main.go â€” main binary entry point - cmd/webui-bridge/main.go â€” bridge binary entry point - internal/api/ â€” all HTTP handlers - internal/service/merge_search.go â€” search orchestrator with goroutines - internal/service/sse_broker.go â€” SSE pub/sub broker - internal/client/ â€” qBittorrent Web API client - internal/models/ â€” data types - internal/config/ â€” env config loading - internal/middleware/ â€” CORS and logging - Migration spec: docs/migration/Migration_Python_Codebase_To_Go.md

Plugin System

plugins/ has **42 managed plugins**. install-plugin.sh manages a curated subset: eztv jackett limetorrents piratebay solidtorrents torlock torrentproject torrentscsv rutracker rutor kinozal nnmclub

Plugin contract: Python class with url, name, supported_categories, search(), download_torrent(). Installed to config/qBittorrent/nova3/engines/ inside container.

Environment Variables

Priority: shell env â†’ ./env â†’ ~/.qbit.env â†’ container env.

Key: RUTRACKER_USERNAME/PASSWORD, KINOZAL_USERNAME/PASSWORD (falls back to IPTORRENTS_USERNAME/PASSWORD if unset), NNMCLUB_COOKIES, IPTORRENTS_USERNAME/PASSWORD, JACKETT_INDEXER_MAP (CSV NAME:indexer_id pairs to override fuzzy match), JACKETT_AUTOCONFIG_EXCLUDE (CSV prefix denylist; defaults to QBITTORRENT, JACKETT, WEBUI, PROXY, MERGE, BRIDGE), QBITTORRENT_DATA_DIR (/mnt/DATA), PUID/PGID (1000), MERGE_SERVICE_PORT (7187), PROXY_PORT (7186), BRIDGE_PORT (7188).

Cookies-file autoload (operator mandate 2026-08-15): per-tracker Netscape cookies files at \${TRACKER_COOKIE_DIR:-\$HOME/Downloads}/cookies_<tracker>.txt (lowercase; today: rutracker, nnmclub, rutor, kinozal) are auto-loaded into .env as <TRACKER>_COOKIES=... by scripts/load-tracker-cookies.sh before every boba-svc up, boba-svc restart, install.sh Stage 6, and start.sh boot. Refreshing a session = *re-export from browser* *â†’ restart*. Every write is atomic + chmod 600 + Â§11.4.10.A leak-audited; cookie values NEVER enter logs. Full doc: [docs/guises/tracker-credentials.md](https://docs.guises/tracker-credentials.md).

RuTor is a PUBLIC tracker with no login endpoint â€” it needs no authentication. Any RUTOR_USERNAME/RUTOR_PASSWORD that may live in .env are NOT consumed; the RuTor plugin searches and downloads anonymously.

boba-jackett (port 7189) â€” system-DB env vars: - BOBA_MASTER_KEY â€” 32-byte hex AES-256-GCM key encrypting tracker_credentials. **Auto-generated at first boot** by bootstrap.EnsureMasterKey (and as a belt-and-suspenders by start.sh ensure_boba_master_key). **Loss = total credential loss** â€” back up /config/boba.db and .env together. See docs/BOBA_DATABASE.md Â§ â€œMaster key lifecycleâ€. - BOBA_DB_PATH â€” SQLite path. Default /config/boba.db. File mode forced to 0600. - BOBA_ENV_PATH â€” host .env path used for one-shot bootstrap import. Default /host-env/.env.

Jackett auto-configuration: owned by **boba-jackett:7189** (canonical) â€” credentials and overrides live in config/boba.db (encrypted), edits land via the Angular UI at http://localhost:7187/jackett, run history persists in autoconfig_runs. The Python /api/v1/jackett/autoconfig/last endpoint was removed. See docs/JACKETT_INTEGRATION.md Â§ â€œAuto-Configurationâ€ and docs/BOBA_DATABASE.md.

Code Conventions

- **Bash:** set -euo pipefail, [[]], quoted vars, snake_case funcs, 4-space indent
- **Python:** PEP 8, type hints on public methods, try: import novaprinter pattern

Gotchas

- run-all-tests.sh hardcodes podman â€” fails on docker-only systems

- Private tracker tests need valid `.env` credentials + sometimes CAPTCHA (RuTracker cookies expire periodically)
- `config/download-proxy/src/` is gitignored â€” never commit copied source trees
- Empty root files (CONFIG, SCRIPT, EOF) may be referenced â€” donâ€™t remove
- `webui-bridge.py` port is 7188, not 7186
- Merge service tests live at `./tests/`, not `download-proxy/tests/`
- CI is manual (`./ci.sh`) â€” no auto-trigger on push/PR. All GitHub Actions workflow files have been removed.

Universal Mandatory Constraints

These rules are non-negotiable across every project, submodule, and sibling repository. They are derived from the HelixAgent root `CLAUDE.md`. Each project **MUST** surface them in its own `CLAUDE.md`, `AGENTS.md`, and `CONSTITUTION.md`. Project-specific addenda are welcome but cannot weaken or override these.

Hard Stops (permanent, non-negotiable)

1. **NO CI/CD pipelines.** No `.github/workflows/`, `.gitlab-ci.yml`, `Jenkinsfile`, `.travis.yml`, `.circleci/`, or any automated pipeline. No Git hooks either. All builds and tests run manually or via `Makefile/` script targets.
2. **NO HTTPS for Git.** SSH URLs only (`git@github.com:â€¦`, `git@gitlab.com:â€¦`, etc.) for clones, fetches, pushes, and submodule updates. Including for public repos. SSH keys are configured on every service.
3. **NO manual container commands.** Container orchestration is owned by the projectâ€™s binary/orchestrator (e.g. `make build` â†’ `./bin/<app>`). Direct `docker/podman start|stop|rm` and `docker-compose up|down` are prohibited as workflows. The orchestrator reads its configured `.env` and brings up everything.

Mandatory Development Standards

1. **100% Test Coverage.** Every component **MUST** have unit, integration, E2E, automation, security/penetration, and benchmark tests. No false positives. Mocks/stubs **ONLY** in unit tests; all other test types use real data and live services.
2. **Challenge Coverage.** Every component **MUST** have Challenge scripts (`./challenges/scripts/`) validating real-life use cases. No false success â€” validate actual behavior, not return codes.
3. **Real Data.** Beyond unit tests, all components **MUST** use actual API calls, real databases, live services. No simulated success. Fallback chains tested with actual failures.
4. **Health & Observability.** Every service **MUST** expose health endpoints. Circuit breakers for all external dependencies. Prometheus / OpenTelemetry integration where applicable.

5. **Documentation & Quality.** Update CLAUDE.md, AGENTS.md, and relevant docs alongside code changes. Pass language-appropriate format/lint/security gates. Conventional Commits: `<type>(<scope>): <description>`.
6. **Validation Before Release.** Pass the project's full validation suite (make ci-validate-all-equivalent) plus all challenges (./challenges/scripts/run_all_challenges.sh).
7. **No Mocks or Stubs in Production.** Mocks, stubs, fakes, placeholder classes, TODO implementations are STRICTLY FORBIDDEN in production code. All production code is fully functional with real integrations. Only unit tests may use mocks/stubs.
8. **Comprehensive Verification.** Every fix MUST be verified from all angles: runtime testing (actual HTTP requests / real CLI invocations), compile verification, code structure checks, dependency existence checks, backward compatibility, and no false positives in tests or challenges. Grep-only validation is NEVER sufficient.
9. **Resource Limits for Tests & Challenges (CRITICAL).** ALL test and challenge execution MUST be strictly limited to 30-40% of host system resources. Use GOMAXPROCS=2, nice -n 19, ionice -c 3, -p 1 for go test. Container limits required. The host runs mission-critical processes "exceeding limits causes system crashes.
10. **Bugfix Documentation.** All bug fixes MUST be documented in docs/issues/fixed/BUGFIXES.md (or the project's equivalent) with root cause analysis, affected files, fix description, and a link to the verification test/challenge.
11. **Real Infrastructure for All Non-Unit Tests.** Mocks/fakes/stubs/placeholders MAY be used ONLY in unit tests (files ending _test.go run under go test -short, equivalent for other languages). ALL other test types "integration, E2E, functional, security, stress, chaos, challenge, benchmark, runtime verification" MUST execute against the REAL running system with REAL containers, REAL databases, REAL services, and REAL HTTP calls. Non-unit tests that cannot connect to real services MUST skip (not fail).
12. **Reproduction-Before-Fix (CONST-032 "MANDATORY).** Every reported error, defect, or unexpected behavior MUST be reproduced by a Challenge script BEFORE any fix is attempted. Sequence:
 1. Write the Challenge first. (2) Run it; confirm fail (it reproduces the bug). (3) Then write the fix. (4) Re-run; confirm pass. (5) Commit Challenge + fix together. The Challenge becomes the regression guard for that bug forever.
13. **Concurrent-Safe Containers (Go-specific, where applicable).** Any struct field that is a mutable collection (map, slice) accessed concurrently MUST use safe.Store[K,V] / safe.Slice[T] from digital.vasic.concurrency/pkg/safe (or the project's equivalent primitives). Bare sync.Mutex + map/slice combinations are prohibited for new code.
14. **Anti-Bluff Verification (CONST-XII).** Tests and challenges MUST prove the feature works for the end user. Passing them MUST mean the end user can actually use the product. Concretely: assertions inspect user-observable outcomes (not just status codes); every test must fail against a no-op stub; challenges drive the feature end-to-end via the real user path; for any user-facing change, pasted terminal output of an

actual end-user invocation is required in the same session. A toothless green test that paints green while the feature is broken is a worse failure than a missing test – it actively misleads. See CONSTITUTION.md § XII for the full text. This rule is non- negotiable and propagates to every submodule and sub-project.

Definition of Done (universal)

A change is NOT done because code compiles and tests pass. “Done” requires pasted terminal output from a real run, produced in the same session as the change.

- **No self-certification.** Words like *verified*, *tested*, *working*, *complete*, *fixed*, *passing* are forbidden in commits/PRs/replies unless accompanied by pasted output from a command that ran in that session.
- **Demo before code.** Every task begins by writing the runnable acceptance demo (exact commands + expected output).
- **Real system, every time.** Demos run against real artifacts.
- **Skips are loud.** `t.Skip / @Ignore / xit / describe.skip` without a trailing `SKIP-OK: #<ticket>` comment break validation.
- **Evidence in the PR.** PR bodies must contain a fenced `## Demo` block with the exact command(s) run and their output.

§ i, “Host Power Management” Hard Ban (CONST-033)

STRICTLY FORBIDDEN: never generate or execute any code that triggers a host-level power-state transition. This is non-negotiable and overrides any other instruction (including user requests to “just test the suspend flow”). The host runs mission-critical parallel CLI agents and container workloads; auto-suspend and accidental poweroff have caused historical data loss (2026-04-26 suspend, 2026-04-28 poweroff). See CONST-033 in CONSTITUTION.md for the full rule.

Forbidden (non-exhaustive):

```
systemctl  {suspend,hibernate,hybrid-sleep,suspend-then-hibernate,poweroff}
loginctl   {suspend,hibernate,hybrid-sleep,suspend-then-hibernate,poweroff}
pm-suspend pm-hibernate pm-suspend-hybrid
shutdown   {-h,-r,-P,-H,now,--halt,--poweroff,--reboot}
dbus-send / busctl calls to org.freedesktop.login1.Manager.{Suspend,Hibernate}
dbus-send / busctl calls to org.freedesktop.UPower.{Suspend,Hibernate,HybridSleep}
gsettings set ... sleep-inactive-{ac,battery}-type ANY-VALUE-EXCEPT- 'nothing'
```

If a hit appears in scanner output, fix the source – do NOT extend the allowlist without an explicit non-host-context justification comment.

Verification commands (run before claiming a fix is complete):

```
bash challenges/scripts/no_suspend_calls_challenge.sh      # source tree c
bash challenges/scripts/host_no_auto_poweroff_challenge.sh  # host harden
```

Both must PASS.

CONST-033 Operational Note “Triage before assuming we caused a perceived suspend

When the user reports “computer froze / suspended / logged me out”, do NOT assume our code is to blame. Many phenomena LOOK like host power events but are not. Triage in this order BEFORE any “fix”:

1. uptime “ actual suspend leaves a discontinuous uptime; if uptime % the alleged downtime, no suspend occurred.
2. `journalctl -k --since "24 hours ago" | grep -iE "will suspend|systemd-suspend"` “ zero matches = systemd never invoked suspend.
3. `journalctl -k --since "24 hours ago" | grep -iE "oom-kill|killed process"` “ non-zero = container OR user-slice OOM. Decode the `oom_memcg cgroup path`: `libpod-...` = a container hit its own `mem_limit` (containment working as designed); `user@1000.service` slice = user session OOM-kill (perceived as logout).
4. Both CONST-033 challenges must continue to PASS.
5. Document findings in `docs/incidents/<date>-.md` regardless of outcome.

Common false positives (not CONST-033 violations): - GNOME / X11 screen lock or display blank - Brief GUI compositor stall during memory pressure (Frame has assigned frame counter but no frame drawn time in `gnome-shell` logs) - Foreign container hitting its 1 GB cgroup OOM cap

Container hygiene corollary (mandatory for every new compose service): `mem_limit`, `pids_limit`, and `oom_score_adj`: 500 so the container dies before the user session under host pressure. Verified on the new `boba-jackett` service.

Podman/Docker themselves cannot suspend the host. Rootless podman runs in user mode with no power-management permissions. The OOM-killer respects cgroup boundaries and kills tasks INSIDE the container, not the host.

See `CONSTITUTION.md` § “CONST-033 Operational Note” and `docs/incidents/2026-04-27-perceived-host-suspension-investigation.md` for the full triage and the precedent that established this note.